The Open University

TM354: Software Engineering

Block III: From architecture to product

Unit 12 The case study: part 3

Unit aims

- This unit is the final part of the case study in the module.
- We consider the overall structure (or architecture) of the case study's system software.
- In particular, we look at how the software that implements the model given in Unit 8 fits with a user interface.
- We see how the system is implemented and we look at how to select unit and system tests.
- The website activities followed are carried through for a small partition of the whole system.
- Only limited functionality is implemented, to produce quickly an initial working system that can be used as a prototype, to allow us to confirm with users some of their requirements, and make refinements where appropriate.
- The unit concludes by looking at some issues that would need to be addressed in the next iteration of development.

Note that: You Are required to practice the Activity Projects in this unit.

Section 1: Introduction

- In this final case study unit we will take the design model of the hotel system developed in Block 2 Unit 8 and investigate its implementation.
- We will begin by looking at three views of the system:
  - the logical view, which describes the system's main functional elements and their interactions; broadly, the services the system provides to users.
  - the process view, which describes independently executing processes that will exist at run-time and the communication between them
  - the deployment view, which describes how the system will be deployed to an operating environment of physical computers and networks.
- We will use some of the models developed in the earlier case study units and add some new ones.
- We will then look at how to verify and validate the implementation.
- We end the unit by exploring some of these: identifying issues that should be addressed in the next iteration and outlining what form the solutions are likely to take.

2.1 The logical view

- The logical view describes the functional elements, how they interact, and how the system can be partitioned into units that can be allocated to teams of developers.

- Typical artefacts are:
  - **structural models** (class and object diagrams), which describe functional elements
  - **behavioural models** (interaction diagrams), which describe interactions
  - **package diagrams**, which describe a partition of the system into loosely coupled chunks that can be developed relatively independently.

Section 2: Architectural views of the hotel system

- Block 2 Unit 8 focused on four closely related use cases:
  - *make reservation*
  - *cancel reservation*
  - *check in guest*
  - *check out guest.*

- The one studied in the most detail was make reservation.

- We analysed it using **object diagrams** to illustrate the state of the system at various points, and developed a design for the interaction involved in this use case, as shown in Figure 3.

- Another type of model that belongs in the logical viewpoint is the **package diagram**, which you met in Block 2 Unit 7.

Section 2: Architectural views of the hotel system

- Figure 2 shows the **class diagram** that was developed in Block 2 Unit 8.

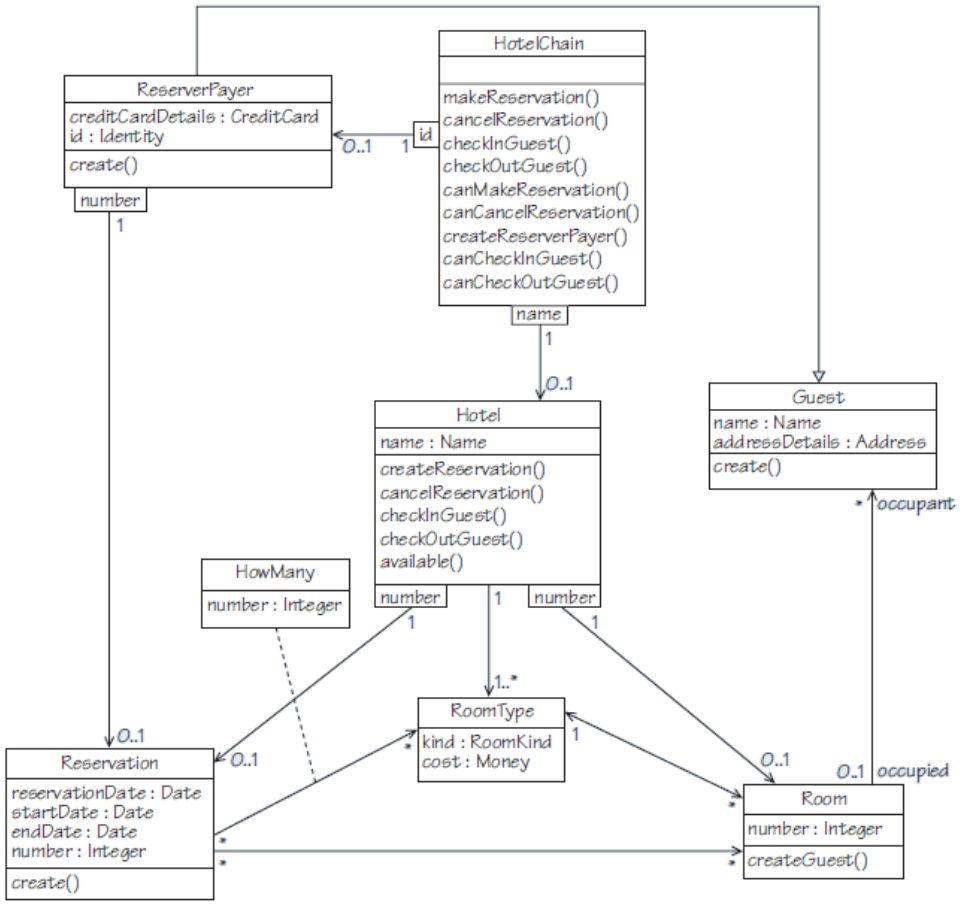

Figure 2 Class diagram with operations

Section 2: Architectural views of the hotel system

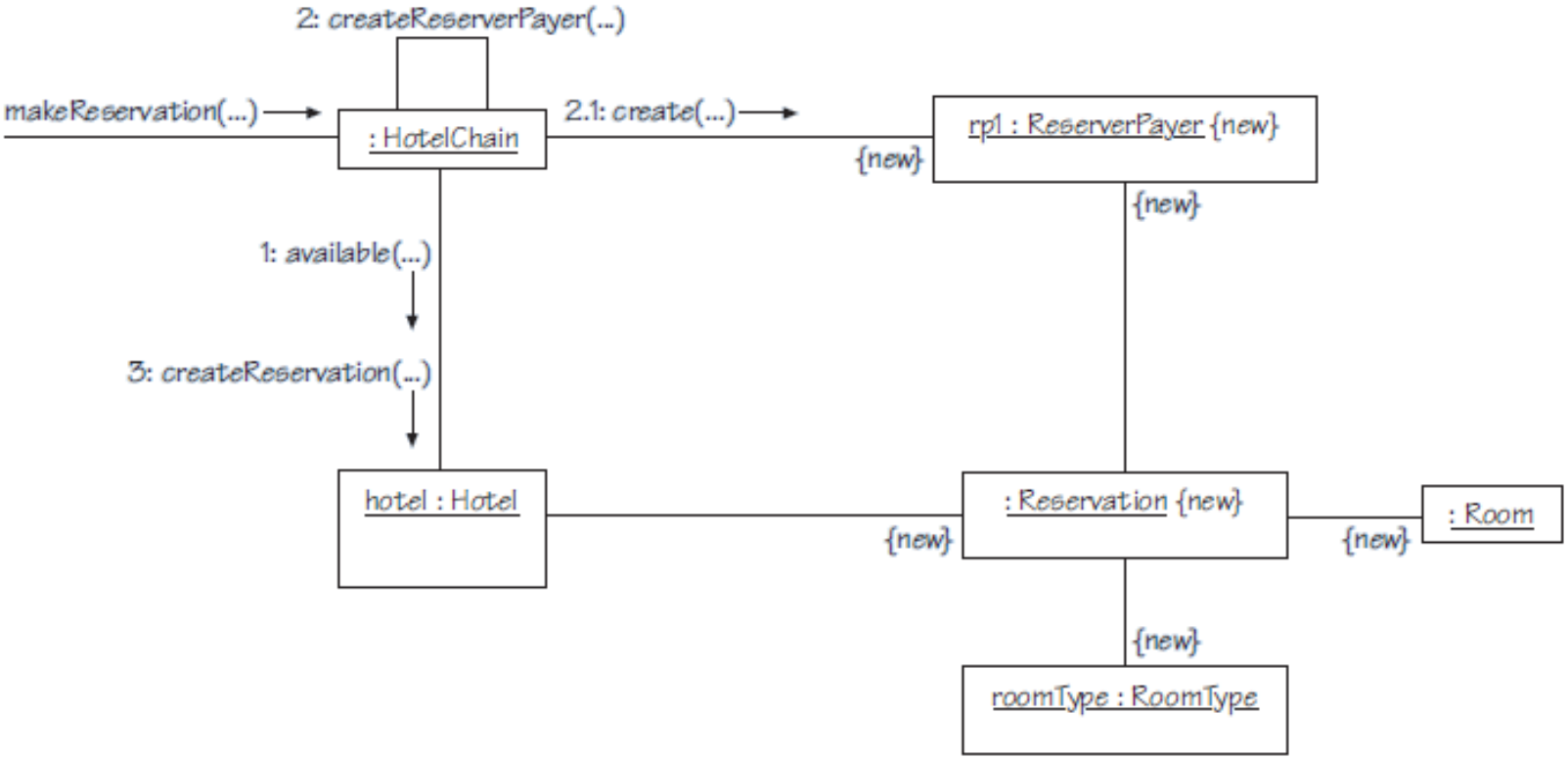

Figure 3 Interaction diagram for *make reservation*

Section 2: Architectural views of the hotel system

- We will assume that the structural model shown in Figure 2 is simple enough for its classes to form a single such unit, which we will assign to a package Model (Figure 4).

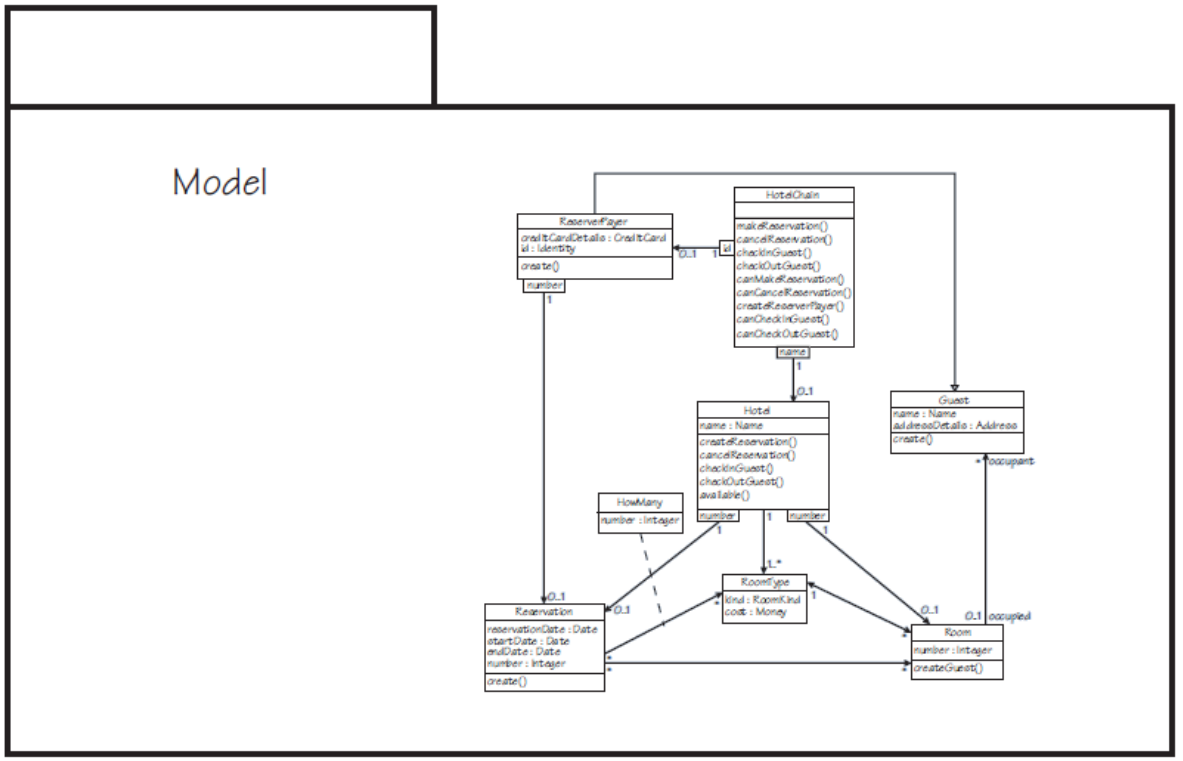

Figure 4 Package *Model*

Section 2: Architectural views of the hotel system

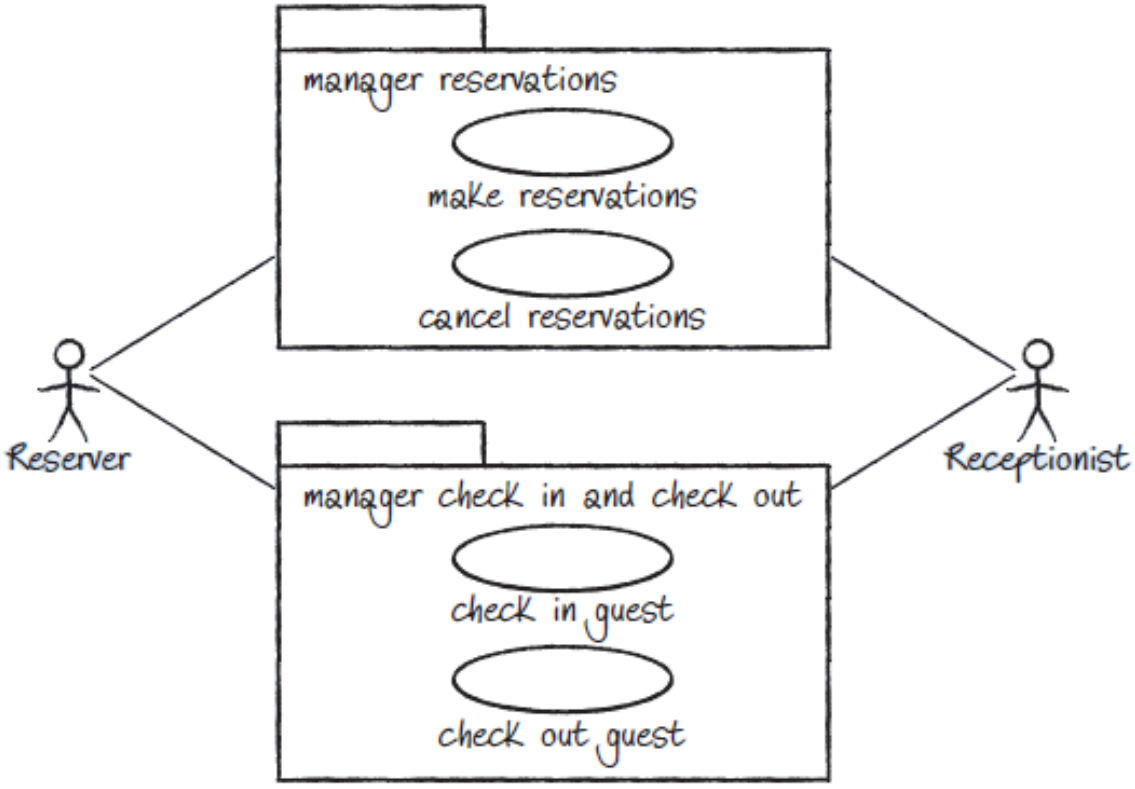

Figure 5 Use cases grouped into packages

Section 2: Architectural views of the hotel system

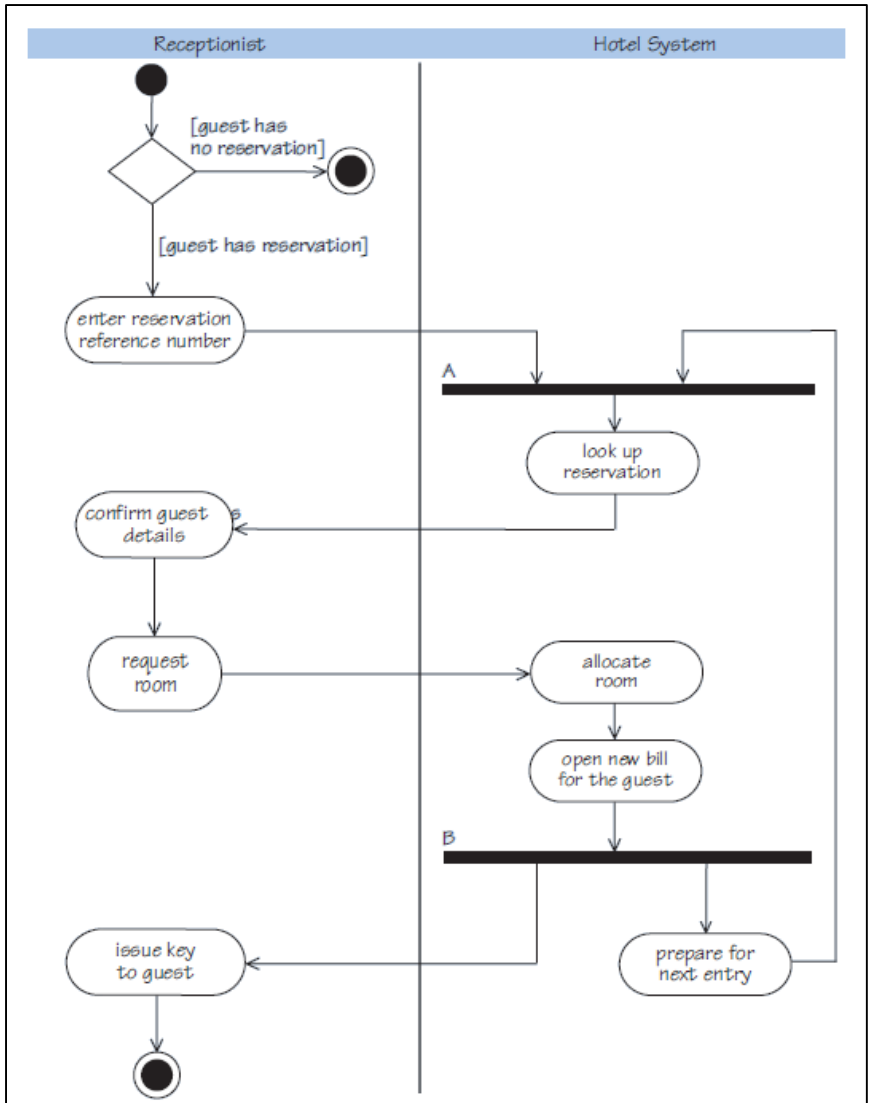

Figure 6 Activity diagram for *check in guest*

2.2 The process view

- The process view is concerned with the separate processes that execute when the system is running and how they communicate and synchronise their actions.
- We can give a simple illustration of the behaviour of processes using an activity diagram, one of the model types used in the process viewpoint.
- Figure 6 is an activity diagram in which a receptionist interacts with the hotel system to fulfil the check in guest use case.

2.3 The deployment view

- The deployment view is concerned with how the system will be deployed to an operating environment.
- We can describe deployment with a deployment model, using a UML diagram which depicts how the high-level components of the system are deployed to physical computers and how the components communicate via networks.
- Figure 7 shows a possible deployment for the hotel system.

Section 2: Architectural views of the hotel system

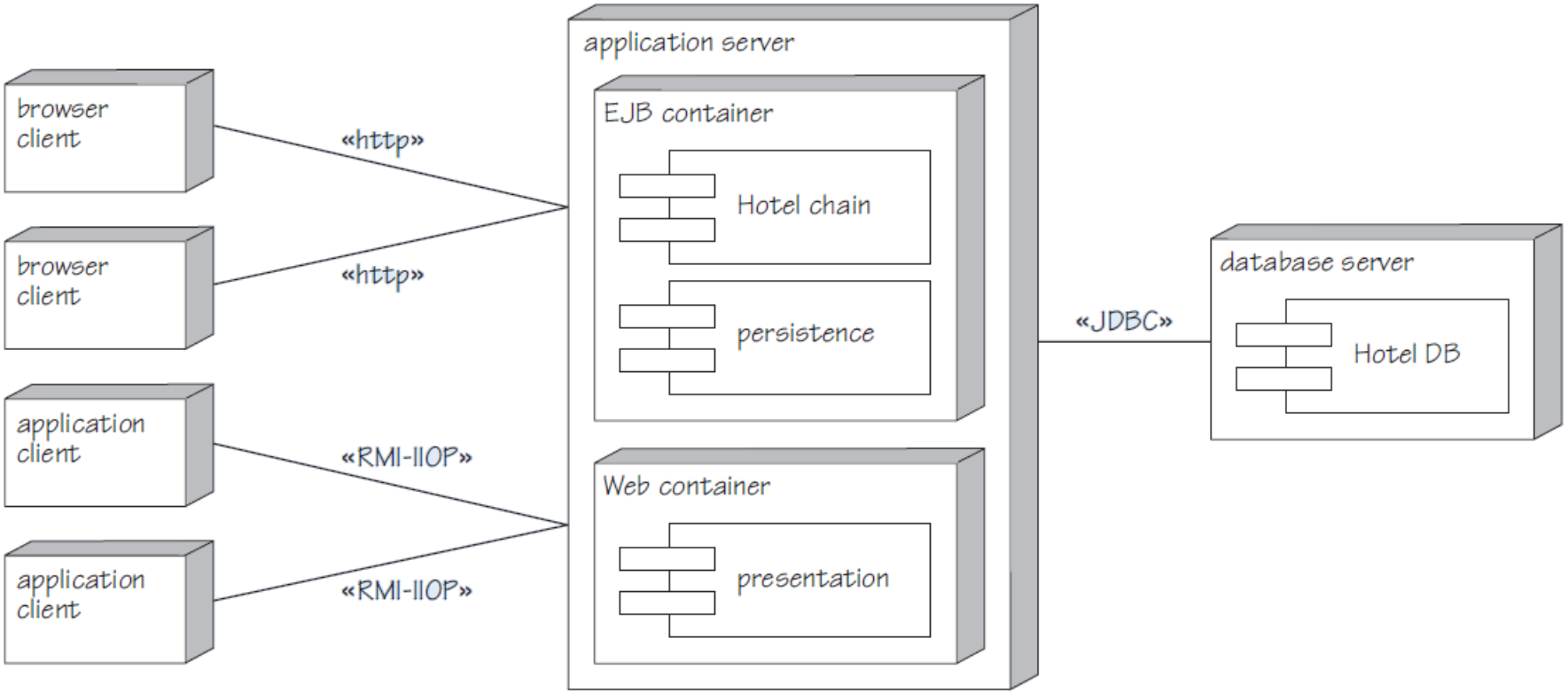

Figure 7 Deployment diagram

2.4 Summary of section

- This section introduced a subset of the hotel system consisting of the four use cases that Block 2 Unit 8 focused on.
- You saw three architectural views of the system:
  - the logical view, which describes the main functional elements
  - the process view, which describes the executing processes that will exist at run-time
  - the deployment view, which describes how the system will be deployed to physical computers and networks.
- For each view some typical artefacts were discussed.
- Artefacts found in the logical view include class and object diagrams, interaction diagrams and package diagrams.
- An example of an artefact from the process view is an activity diagram.
- A typical artefact from the deployment view is a deployment diagram.

3.1 Choosing an architecture

- We want the user interface to be as independent as possible from the model, so we will use a design pattern that we haven't discussed yet, the Façade pattern (Gamma et al., 1995).
- In built architecture a façade is a frontage for a structure, the face it presents to the world.
- The software design pattern gets its name from the same idea.
- If a client needs to deal with a number of different objects, they can all be hidden behind a Facade object, which is like a sort of one-stop shop.
- This makes things simpler for the client, because it only needs to know about the Facade, which presents it with a single unified interface.

Section 3: Implementing a first iteration

- The **Facade** accepts client requests and forwards them to the appropriate supplier objects.
- This reduces coupling, because the client doesn't need to know anything about the supplier objects, and their implementation can therefore be changed without any impact on the client.
- The Façade can translate between different data formats, allowing the client to work with a different set from the supplier objects.
- Also acts to reduce coupling and helps limit the scope for errors 'leaking' into the model and compromising its integrity.
- The logical view described by Figure 8.

Section 3: Implementing a first iteration

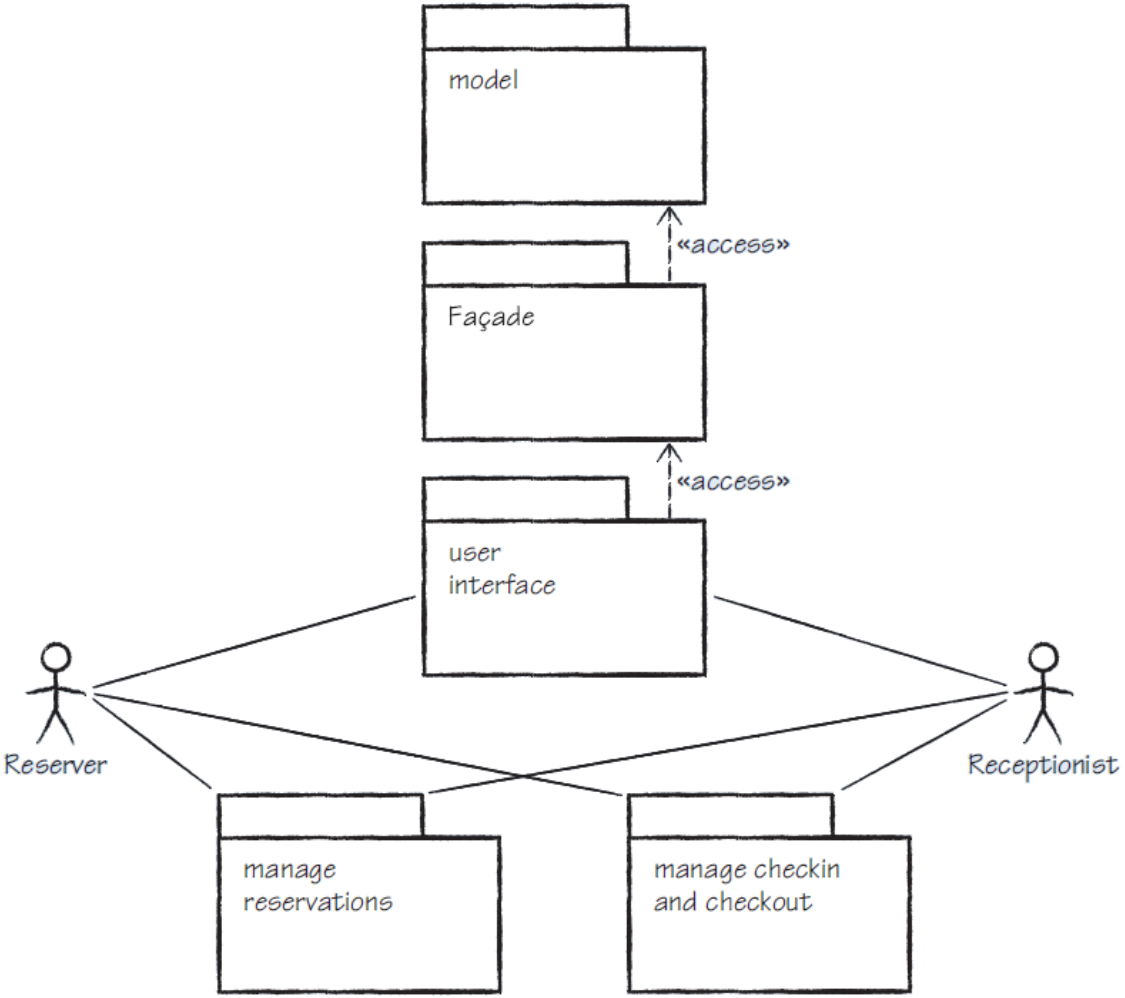

Figure 8 Logical view

Section 3: Implementing a first iteration

- The process and deployment views are extremely simple for this iteration.
- There will be only a single process and there are no issues of synchronization
  - Since we are using a call-return style in which an object that calls an operation on another object must wait for the operation to complete before it can carry out any further actions.
- Deployment is trivial in this iteration since everything takes place on the same computer and within the same Java Virtual Machine shown in Figure 9.
- This diagram shows only the high-level components.

Section 3: Implementing a first iteration

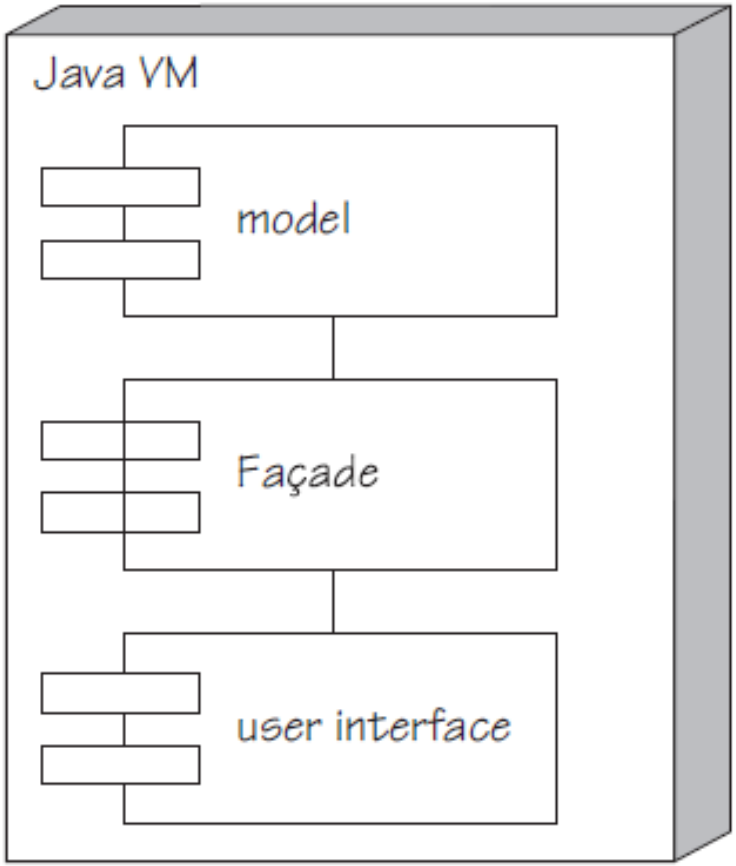

Figure 9 Deployment diagram for this iteration

3.2 Implementing the first iteration

- Before we can implement any use cases we need a first iteration of the structural model.
- In principle any object-oriented programming language could be used but the target language
- Java is used in this module, so the classes will be Java classes.
- The operations will translate into Java methods and constructors

Attributes

- Attributes represent properties of an object and the values of an object's attributes represent its state.
- They are implemented by variables which hold the attribute values.
- If the value of an attribute is the same for all instances of a class it can be implemented as a class variable.
- Otherwise it will be implemented as an instance variable and each instance of the class will have its own value for the variable.
- Attributes can also have visibility.
- In Java, class and instance variables can be given one of four levels of ***visibility:***

Section 3: Implementing a first iteration

- In Java, class and instance variables can be given one of four levels of ***visibility:***
  - private – visible only to the class and objects of the same class
  - package – visible to classes and objects in the same Java package
  - protected – visible to classes and objects in the same Java package and to the class's subclasses and their instances anywhere
  - public – visible to all classes and their instances.

- For private, protected and public visibility, Java provides explicit modifiers. If no visibility is specified it defaults to package access.

- Variables may additionally be declared **final.** Once a value has been assigned to a final variable no other value can subsequently be assigned to it.

Data types

- The attributes used in the model all have data types such as Name, Integer, RoomKind, Money and Date.
- High-level languages come with a range of standard data types built-in and in many cases one of these will be suitable to represent an attribute
- But others, such as RoomKind and Money, cannot sensibly be represented by any existing Java types. For these, user-defined types will be required.
- Analysis pattern called Quantity, which provides useful guidance on how to represent quantities such as amounts of money.

Associations

- Associations are more complex than attributes for several reasons:
  - They involve two classes rather than just one.
  - They have multiplicities. A multiplicity greater than one means a link to a collection of objects rather than just a single object.
  - They may be qualified. If an association is qualified then every link of the association has a unique identifier.

Section 3: Implementing a first iteration

Association end with multiplicity one

- Consider the association from Room to RoomType shown in Figure 11. This association tells us that each Room object must be linked to a RoomType object – that is, the room type of that room.

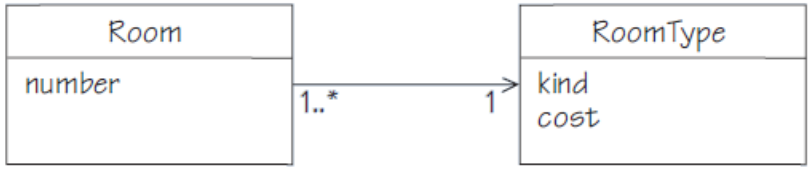

Figure 11 Association end with multiplicity one

- This link will be represented by using an instance variable in the Room class to store a reference to the RoomType instance concerned. This is shown schematically in Figure 12.

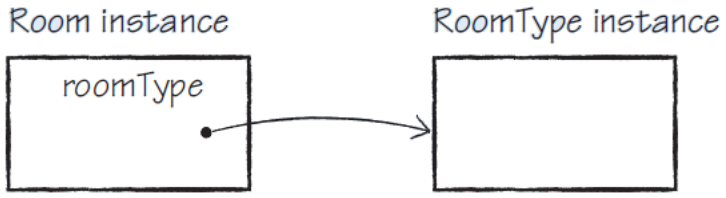

Figure 12 Object reference representing association link

Association end with multiplicity many

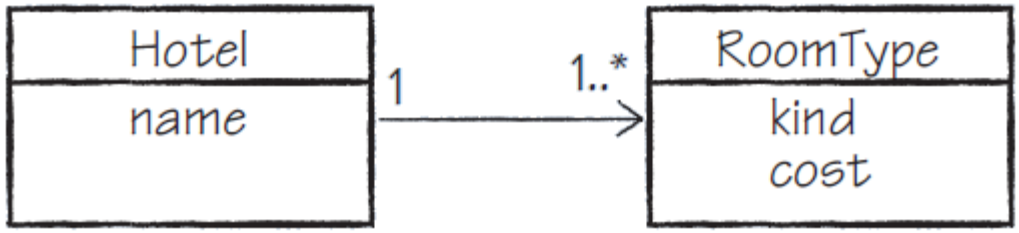

Figure 13 Association end with multiplicity many

Qualified Associations

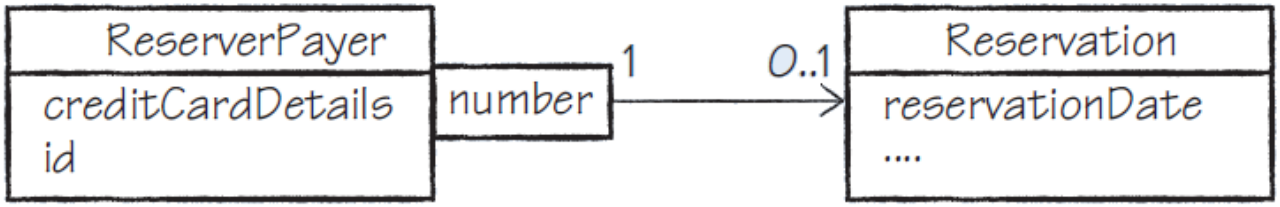

Figure 14 Qualified association

3.3 Implementing and testing operations

- Having considered the implementation of attributes and associations we turn now to implementing operations.

- To get from an operation in the design model to a method that implements that operation involves the following steps:

1. matching the signature and return type
2. writing tests
3. running the tests, which should fail at that point
4. realising the contract of the operation in code
5. rerunning the tests and if one or more fail correcting the code until all the tests are passed.

3.4 Summary of section

- This section considered a first iteration of the system.
- It began by choosing an architecture based on the MVC pattern, which separated the core system from the user interface.
- To further reduce coupling between the two a Façade layer was also introduced.
- The section then went on to explore in some detail how the system can be implemented in Java, in particular how to realise attributes, associations and behaviour.
- Then we have seen the attributes and associations
- Finally we have seen that operations are realised as Java methods.

Section 4: Using the Facade pattern

- Figure 17 shows a Facade hiding details of a core system, including converting between data types, which we will explain in detail below.

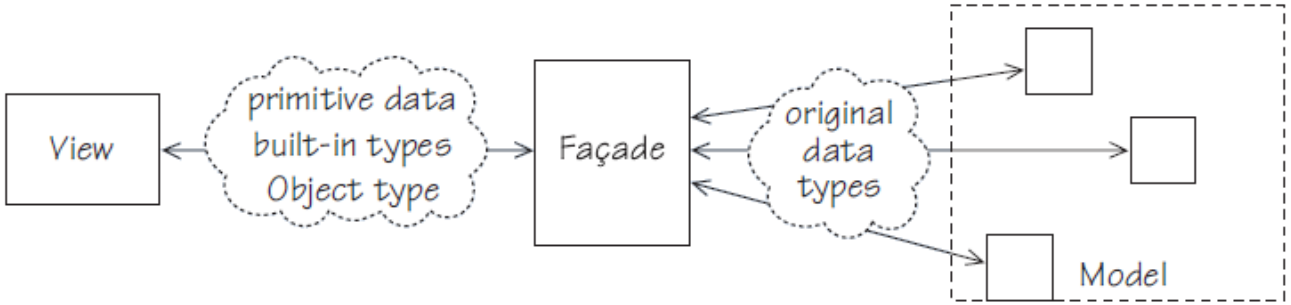

Figure 17 Facade

- In our system we use a Facade mainly to decouple the user interface from the business logic. Two main sorts of operation are required:
  - operations that implement use cases (such as makeReservation)
  - operations that provide the user interface with additional information.

Section 4: Using the Facade pattern

Facade pattern

- *Name*. Facade.
- *Purpose*. Replaces a set of separate interfaces with a single unified interface which is easier for clients to use and which reduces coupling between the client and the supplier objects.
- *How it works*. A *Facade* object sits between the clients and the individual interfaces. The Facade receives client requests and translates them into requests that can be understood by the supplier objects.
- *When to use it*. When we want to present clients with a single interface instead of several, or when we want to reduce the coupling between clients and system objects, for example by allowing clients and supplier objects to vary their data representation independently.
- *Example of use*. An online shop may act as a front for a number of individual suppliers but this is hidden from customers, who interact with a single ordering and payment system. The shop may accept card payments, and convert the amount charged from the currency used where the shop is based into the one used in the customer's country.

Section 4: Using the Facade pattern

- A Facade pattern is used mainly to decouple the user interface from the business logic. Two main sorts of operation are required:
  - operations that implement use cases (such as makeReservation)
  - operations that provide the user interface with additional information.

- To minimise coupling between the user interface and the business layer we want the user interface to know as little as possible about the core system.

- The two sides should trade in different kinds of data.

- At the user interface end the data should be confined to primitive data types, built-in Java utility types such as String, and objects that the user interface knows only as instances of Object (in Java): their exact class is not known to the user interface.

4.1 Summary of section

- This section discussed the Facade design pattern and how it is used in the hotel system.
- You saw that to reduce coupling to a minimum the user interface should be unaware of the actual classes of objects in the business layer.
- To maintain the separation, the user and the interface and the core system should trade in different types of data and the Facade is responsible for the conversions between types that this makes necessary.
- You learnt that two kinds of operation are needed: ones that implement use cases, and ones that provide the user interface with additional information, and you saw an example of each kind.

5.1 Functionality

- This section explains hoe the overall system would be tested.

- **System testing**
  - System testing is testing we carry out to check that the completed system meets its requirements.
- ***Refer to the given example test scenarios***

- **Acceptance testing**
  - Acceptance testing is similar to system testing but carried out by the customer to decide whether to accept or reject the software.
- ***Refer to the given example test scenarios***

5.2 Quality attributes

- In this subsection we give an example of a quality attribute scenario and a set of tactics that can be applied to our hotel system, building on the examples given in Unit 10.
- Figure 7 from Unit 10, quality attribute scenarios based on the six-part mode of the Bass et al six-part model, is repeated here as Figure 18.
- ***Refer to the given examples***

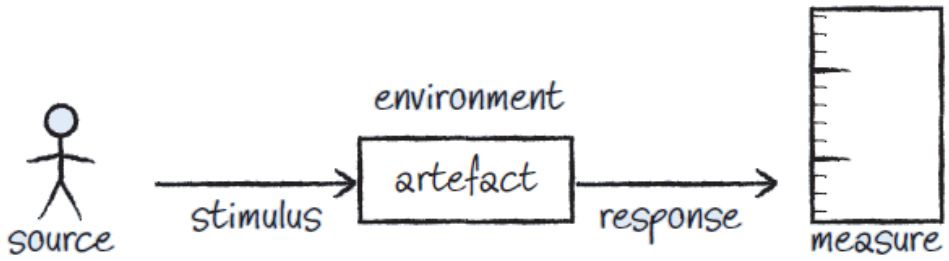

Figure 18 Quality attribute scenario

- **Tactics for flexibility**

- The main tactics for architecture flexibility identified in Unit 10 were:
  - minimise coupling
  - maximise cohesion
  - keep modules small
  - bind as late as possible.

5.3 Summary of section

- This section was concerned with whether the system meets its requirements.
- First you looked at system testing, then looked at acceptance testing
- A quality attribute scenario for usability was developed using the six-part model of Bass et al. introduced in Unit 10, and you then investigated whether the system that has been implemented met the associated measure.
- Tactics are reusable solutions for meeting quality attribute requirements.
- The section ended by exploring how far the architecture chosen for the first iteration supports the quality attribute of flexibility.

6.1 Persistence

- In Java it is possible to save objects in an ordinary file and reload them, but this is not suitable for the kind of large-scale persistence that would be required in an enterprise application.
- For that we need a database.
- To explain how our system could be made persistent it is easiest to begin by outlining how data is stored in a database, and then to explain how the stored data corresponds to the business objects in an application.

Section 5: The way forward

- In a database the data is normally stored in tables.
- For example, data about room types could be held as in Table 8.

Table 8 ROOM_TYPE

ID (integer)	KIND (text)	COST (decimal)
1	SINGLE	80.00
2	DOUBLE	120:00
3	DOUBLE	150.00
…	…	…

- The rows in this table each represent a record, containing the data for one particular room type.
- The columns each contain data of one specific type, such as text or decimal.
- The ID column is special because it uniquely identifies each record.
- A column that uniquely identifies the rows of a database table is called a **primary key.**

Section 5: The way forward

- The table for rooms might look like Table 9.

Table 9 ROOM

ID (integer)	ROOM_TYPE_ID (integer)
101	1
102	1
…	…
201	3
…	…

- This has a column named ID that is the primary key.
- It also has a column ROOM_TYPE_ID that holds keys from a different table.
- This is a **foreign key**.
- Notice that the foreign key enables us to find the room type of any given room

Section 5: The way forward

- Database tables can be used to make business objects persistent
- The business objects can be mapped to database storage in the following way.
  - For each class of business objects there is a corresponding database table.
  - Each instance variable is represented as a column of the table.
  - Each object belonging to the class is represented as a row of the table.
- Figure 19 shows a UML object diagram corresponding to the three rows in Table 9.

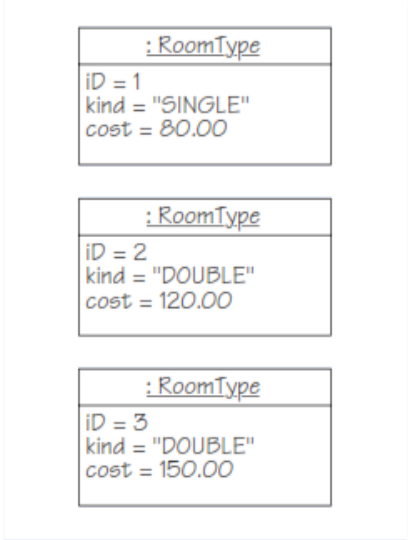

Figure 19 Objects corresponding to database rows

- **6.2 Distribution – Java Server Faces (JSF)**

- JSF is a popular framework for developing browser-based user interfaces.

- JSFs are part of Java EE and run in a web container.

- JSF, like many frameworks, is based on MVC and supports a very clear separation between presentation and business logic.

- Figure 20 gives an overview of how the MVC roles relate to the JSF framework.

Section 5: The way forward

- Figure 20 gives an overview of how the MVC roles relate to the JSF framework.

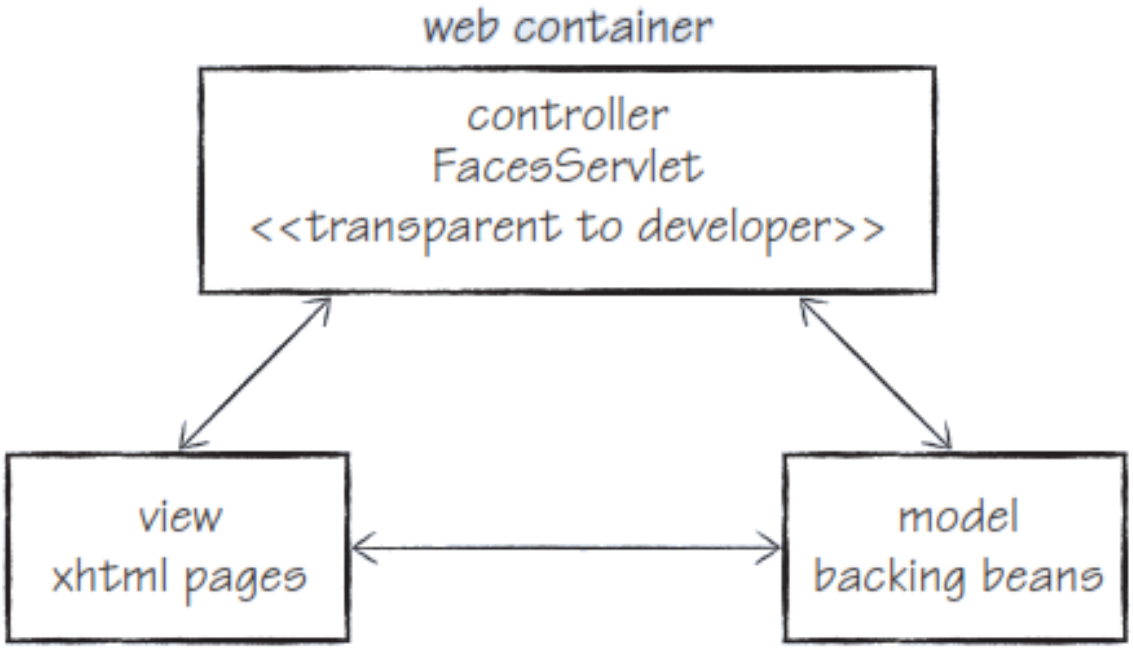

Figure 20 JSF and MVC

Section 5: The way forward

- Figure 21shows a view of the running application as it would appear in a browser window..

Figure 21 The converter displayed in a browser

Controller

- In JSF the role of the controller is taken by a servlet called the Faces Servlet, which is a component generated automatically by the container.
- The Faces Servlet remains behind the scenes and the developer never needs to be aware of it.

Model

- The role of the model is taken by a Java EE component called a managed bean.

View

- The role of the view is taken by a page written in XHTML, a language for defining web pages.

6.3 Summary of section

- In this section you began by looking at a number of different ways in which this first iteration of the hotel system is incomplete.
- Two aspects in particular were identified – the lack of persistence and the fact that the system is not distributed.
- You were given a brief introduction to how data can be made persistent by storing it in database tables, and saw how the JPA allows Java programs to interact with a database.
- You then met JSF, a popular framework based on MVC, and saw how
- Managed beans and XHTML pages might be used to provide the hotel system with a browser-based interface that would allow users to access it via the internet.

Unit Summary

On completion of this unit you should be able to:

- Discuss architectural views and know some of the artefacts typically found in each view
- understand how attributes, associations and operations can be implemented in a language such as Java
- follow how unit, system and acceptance tests can be specified and carried out
- understand the purposes of the Facade software pattern
- appreciate how a quality attribute scenario can be developed and applied
- understand how tactics for quality attributes can be used to look for improvements in an architecture
- understand some ways of reviewing an iteration and planning changes to be made in following iterative cycles.